

視窗程式設計 期末專題

S1154005 李育林

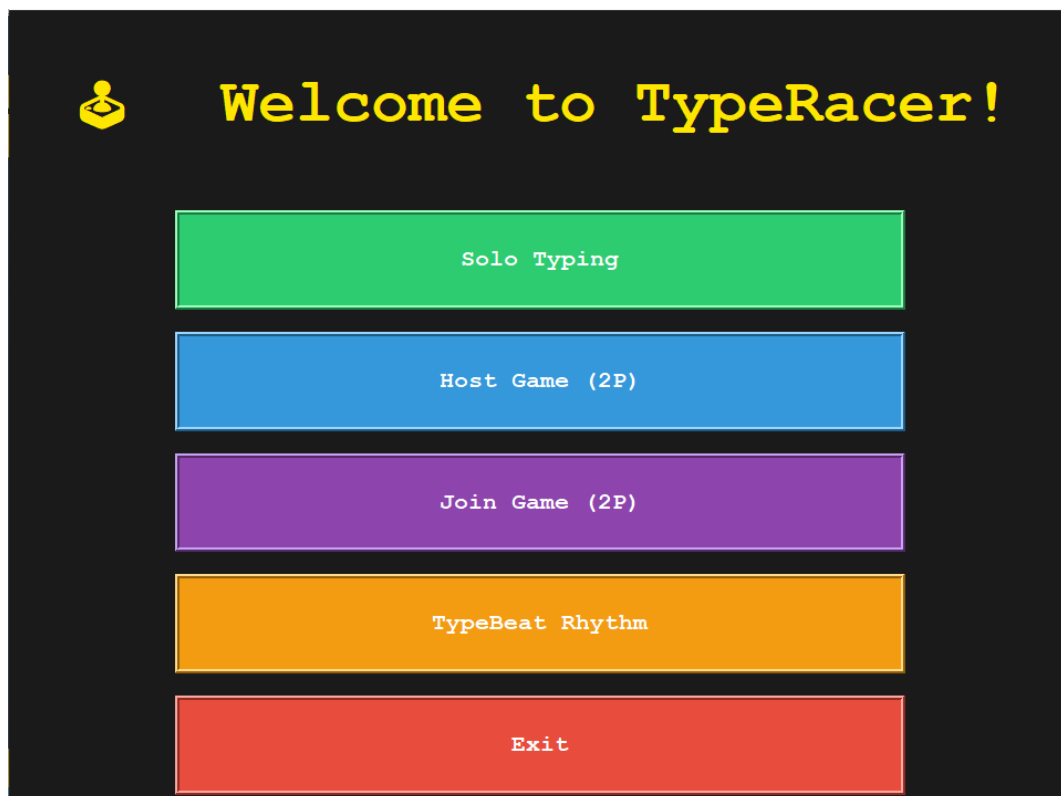
組員：S1154002 陳品璵

目次

- 壹、設計理念
- 貳、使用函式庫
- 參、功能說明
- 肆、困難與解決方法
- 伍、結論
- 陸、程式碼節錄

壹、設計理念

就讀資訊工程系的大家為了成為一位優秀的工程師，每天都在和程式碼奮鬥，隨著大家越來越厲害，程式碼也越來越長，也更加凸顯了打字速度的重要。因此設計了一個打字練習遊戲，提供單人和雙人聯機對戰，並且還有特殊的太鼓練習模式。

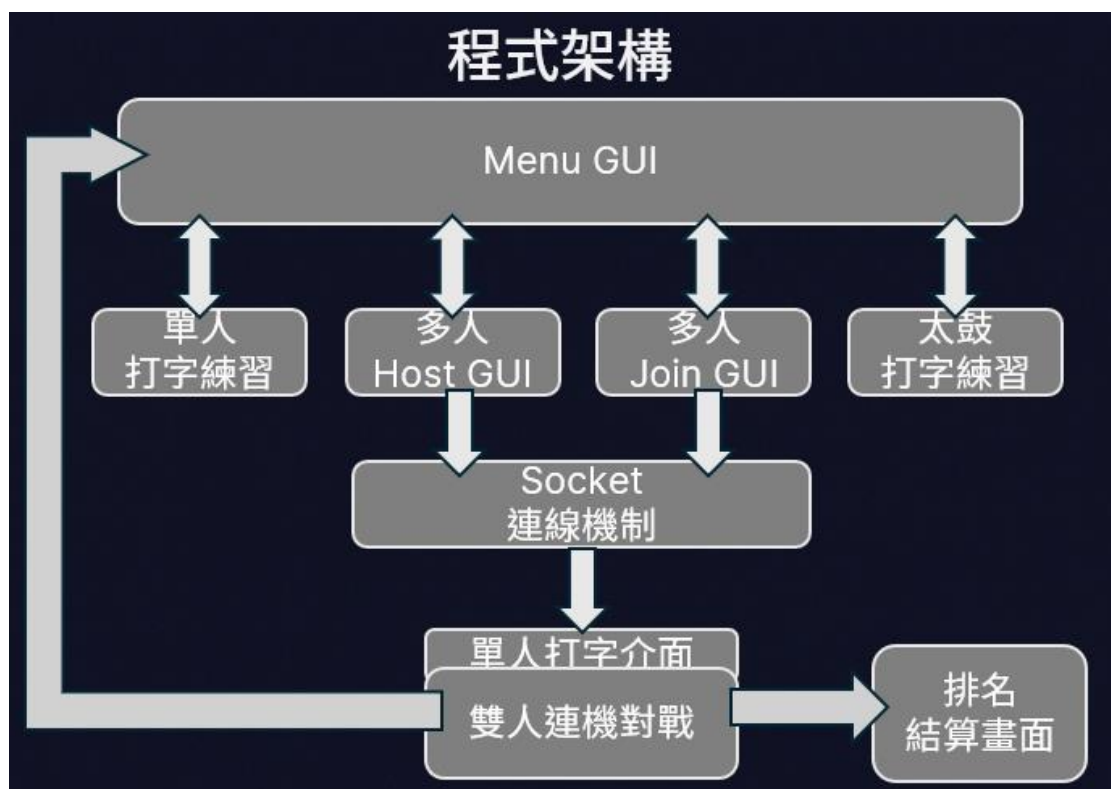


貳、使用函式庫

函式庫	主要功能
Time	打字用時計時
Random	練習文本抽選&太鼓模式音符生成
Socket	建立與管控雙人連線機制
Threading	Socket 監聽&優化程式運行

參、功能說明

本程式主要有三個模式，三個模式分別可透過 Menu 介面做點選選擇要進行之模式(附圖一)，選擇後分別進入相對應之遊戲模式開始進行遊戲。接下來分別介紹三個模式的功能與實作方式，：



圖一、整體程式架構

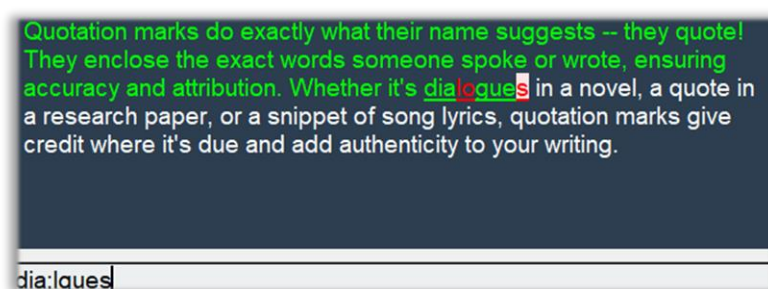
一、單人練習模式

讓使用者透過經典 TypeRacer 模式，練習輸入一段完整英文文章，並測量每分鐘字數（WPM）及準確率且可以計時完成時長。準確率的計算採用單字級比對，當輸入單字正確時會更新，設計上若有重複輸入錯誤並不會重複計算。此外，玩家中途也可以隨時選擇重新抽選文本重新開始或回到標題畫面。

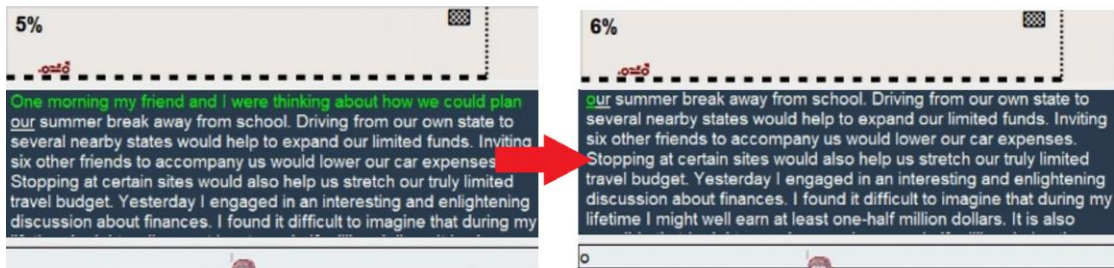


圖二、單人模式介面

在畫面最上方可以看到有一個賽車進度條，為本遊戲最大特色之一，當玩家輸入正確之單字時，賽車就會往終點前進一定距離，當完成文章時賽車就會抵達終點。在畫面中間部分，是文章顯示區域及打字輸入欄位，本遊戲為了提供玩家最優秀的回饋體驗，在輸入方面有加入以下功能：1. 即時字元級別正確錯誤回饋(附圖三) 2. 文章過長超出顯示區塊自動換行(附圖四) 3. 完成目前單字才可繼續打下一個單字。



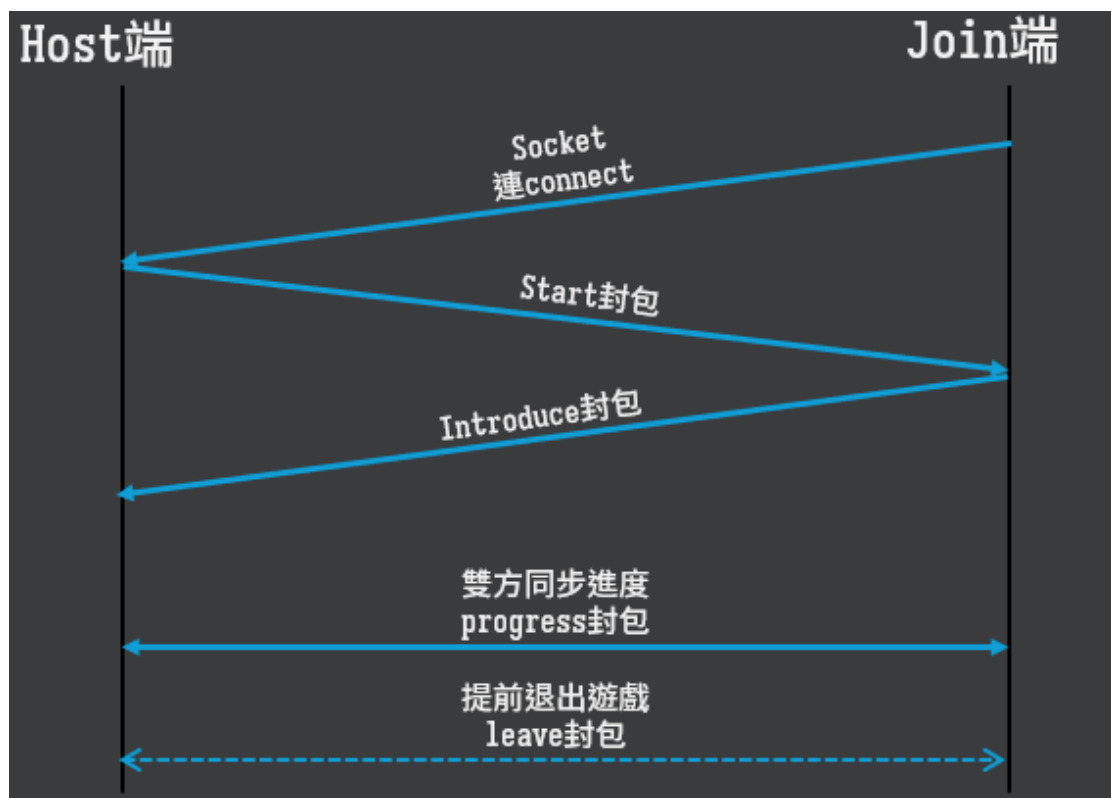
圖三、即時字元輸入回饋(綠色：正確輸入，紅色：錯誤/多於輸入)



圖四、玩家無須手動調整文章，顯示需要換行時會自動判斷

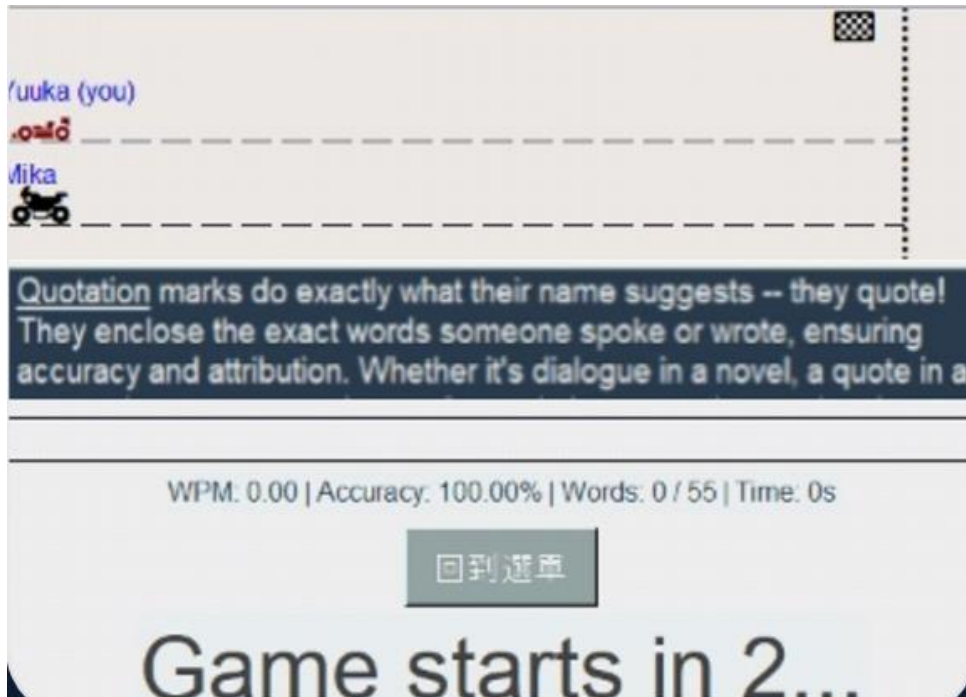
二、雙人聯機對戰模式

提供兩位使用者在區域網路中連線對戰，同時輸入相同的句子並比較輸入速度與準確性。連線方式分為 Host 以及 Join 端，分別使用 Socket 函式庫建立 TCP 通訊，並透過 Threading 方式監聽處理雙方遊戲同步邏輯與封包協定，詳細連線溝通流程如下(附圖五)。



圖五、雙人聯機遊戲 TCP 通訊流程圖

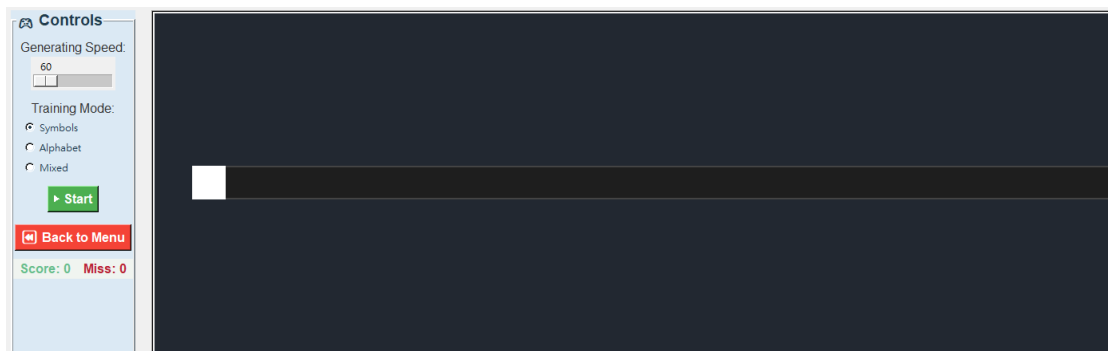
雙方建立連線之後即進入聯機遊戲，畫面最上方的進度條會同步顯示雙方的進度，當一方因斷線或遊戲關閉而退出也會即時告知對方，最後當雙方都完成時，會有一個簡單得結算動畫。而單人模式下有的一些打字輸入回饋和 WPM、準確率等統計也有保留在雙人模式下。



圖六、聯機遊戲之遊戲畫面

三、太鼓打字練習模式

模仿《太鼓達人》節奏輸入玩法，訓練使用者針對符號、大小寫等特殊字元的反應能力，主要是針對打字時常因為大小寫切換或特殊字元的不熟悉導致打錯而延伸出來的練習模式。



圖七、太鼓模式之遊戲畫面

玩家可透過左方控制欄位選擇要練習之模式及音符生成之速度，開始遊戲後右方的遊戲區域會開始生成音符，玩家需要在音符抵達判定區時進行相對應輸入，並會產生 HIT 或 MISS 的動畫回饋，最後當玩家 MISS 三次時遊戲結束。

肆、困難與解決方式

1. 文章長度超出顯示範圍閱讀問題

打字過程，玩家不可能還要手動去滾動文章顯示區塊來查看後續文章內容，因此需設計一個文章內容隨玩家輸入進度自動滾動的機制。當玩家正確輸入目前單字時同時會去判斷剩餘文章內容是否已經全部顯示在可視區域，接著會去比對若要調整可視區文章就會計算目前進度下文章區域應該從哪一段文字開始顯示。判斷部分邏輯較為複雜，研究調整時花了不少時間。

2. 雙人聯機中途斷線與進度同步問題

設計專門處理進度與離線的封包(Progress、Leave 封包)。當玩家完成單字推進進度時就會送出 Progress 封包告知另一方要更新進度，而當玩家中途離開則會送出 Leave 封包告知另一方對手已退出，接著再去做相對應處理機制。

3. 視窗大小拉伸與打字體驗取捨

一開始在設計時都有考慮到視窗大小拉伸的畫面調整，包括進度條和所有其他區域都會根據畫面拉伸去做自適應調整。不過視窗大小的拉伸很容易導致可視文章區的壓縮或是顯示過大導致玩家閱讀困難進而使打字體驗不佳。最終設計上改成視窗大小顯示，玩家可以自由選擇 4:3 或 16:9 的比例進行遊玩。

4. 太鼓模式判定範圍設計

最初實作時沒有過多考慮判定點的設計，會導致同一音符重複判定或是還在很遠時誤觸就判定 MISS，導致遊玩體驗不佳。調整判定 HIT 與 MISS 之範圍，大約音符有接觸到判定點都算是 HIT。

5. 控制邏輯與動畫更新造成 UI 不流暢

在原先設計時，一些動畫(EX: 小車進度條的移動、太鼓模式的音符流動)直接使用函式呼叫方式會導致整個程式明顯卡頓。最後使用‘after’函式，透過 non-blocking 的方式去安排這些動畫的執行。

6. 與組員合作不順

偷偷抱怨一下組員，由於我這次沒有提前去找組員加上大家都以為我找好組了，最後只好匆匆找了一個隊友，抽報告順序又很不幸的抽到了第一週報告，更應該加緊腳步把成品給做出來。不過組員似乎不太把這件事放在心上，也可能是因為我標準比較高，導致這次期末專題幾乎是獨力完成。還是要有好的組員一起努力才開心。

伍、結論

這次透過這門課的機會嘗試製作了一直以來很有興趣但又沒時間去做的遊戲製作，我們這組是第一週報告又加上我太晚找人的原因，實際製作時間非常的短，因此製作出來的內容比較簡單還有很多功能可以再去做添加，像是：排行榜、個人紀錄分析、不僅限於雙人的聯機對戰、更多不同的文章題庫……，不過我覺得也有許多做得不錯且有特色的部分，像是：賽車進度條添加遊玩樂趣體驗、額外的太鼓練習模式能夠針對常錯的大小寫切換和特殊字元練習、在使用者打字體驗上也有特別去思考琢磨。整體而言，我們製作了一個有趣又實用的打字練習程式，在製作上特別針對了使用者的打字體驗和趣味性去做設計實作，是一個很好的機會讓大家發揮一下想像力去做各種東西和遊戲。

伍、程式碼節錄

Source Code:

[WindowProgramming/final_project at main · DieWay0123/WindowProgramming](#)

文章顯示區域畫面渲染(即時回饋輸入正確/錯誤)

```

# current_input: 目前單字輸入的位置(目前進行中的單字)
def render_article(self, current_iunput=""):
    # 重新渲染文本框前要先將舊資訊清除
    self.article_display.config(state="normal")
    self.article_display.delete("1.0", tk.END)
    for tag in self.article_display.tag_names():
        self.article_display.tag_remove(tag, "1.0", tk.END)

    for i, word in enumerate(self.target_words):
        for j, char in enumerate(word):
            tag = "default"
            # 處理已經完成的單字文本顯示
            if i < len(self.typed_words):
                if j < len(self.typed_words[i]):
                    if self.typed_words[i][j] == char:
                        tag = "correct"
                    else:
                        tag = "incorrect"
                else:
                    tag = "incorrect"
            # 處理目前進行到的單字文本顯示
            elif i == self.current_word_idx:
                if j < len(current_iunput):
                    if current_iunput[j] == char:
                        tag = "current_correct"
                    else:
                        tag = "current_incorrect"
                else:
                    tag = "current_pending"
            self.article_display.insert(tk.END, char, tag)

        # 當目前單字多餘輸入時同樣在文本上顯示錯誤
        if i == self.current_word_idx and len(current_iunput) > len(word):
            extra_text = current_iunput[len(word):]
            for c in extra_text:
                self.article_display.insert(tk.END, c, "extra_wrong")
            self.article_display.insert(tk.END, " ", "default") # 不要忘了單字間的空格
    self.article_display.config(state="disabled")
    self.article_display.yview_scroll(self.line, "units")

```

文章可視自動滾動


```

def auto_scroll_after_first_line_done(self):
    text = self.article_display
    typed = self.typed_words

    # 若不需要調整
    bbox = self.article_display.bbox("end-1c")
    if bbox:
        x, y, w, h = bbox
        if y+h <= self.article_display.winfo_height() - 20:
            return

    # 從index "1.0" 開始逐字找第一行
    line_end_idx = None
    prev_y = None
    for offset in range(len(self.target_article)):
        idx = f"1.0 + {offset} chars"
        bbox = text.bbox(idx)
        if not bbox:
            continue #!FIX
        x, y, w, h = bbox
        if prev_y is None:
            prev_y = y
        elif y != prev_y:
            line_end_idx = f"1.0 + {offset - 1} chars"
            break
    else:
        line_end_idx = f"1.0 + {len(self.target_article) - 1} chars"

    # 取得該位置的字元並向前尋找該字屬於哪個單字
    char_pos = text.index(line_end_idx)
    linear_index = int(char_pos.split('.')[1]) # 因為整段是 1 行
    word_end = 0
    for i, word in enumerate(typed):
        word_end += len(word) + 1
        if linear_index + 1 == word_end:
            self.line += 1
            break

```

綁定輸入事件進行字元級別的即時輸入正確/錯誤偵測

```

def on_key_release(self, event):
    current_input = self.typing_entry.get()

    # 計算輸入錯誤次數
    if self.current_word_idx < len(self.target_words):
        target_word = self.target_words[self.current_word_idx]
        is_wrong = False
        # 判斷目前輸入與目前打到的文本單字是否有錯誤
        for j in range(min(len(current_input), len(target_word))):
            if current_input[j] != target_word[j]:
                is_wrong = True
                break
        if len(current_input) > len(target_word):
            is_wrong = True
        if is_wrong and not self.has_made_mistake:
            self.total_wrong_words += 1
            self.has_made_mistake = True

    self.render_article(current_input=current_input)
    if not self.is_typing:
        self.start_time = time.time()
        self.is_typing = True
        # 暫時關閉menu不能更改尺寸
        self.root.config(menu="")

```

綁定空白鍵事件作為當前輸入單字之判斷(單字間距皆為空格)

```

def on_space_press(self, event):
    typed_word = self.typing_entry.get().strip()
    if self.current_word_idx < len(self.target_words):
        # 若目前打的單字完全正確可以進入下一個單字
        if typed_word == self.target_words[self.current_word_idx]:
            self.typed_words.append(typed_word)
            self.current_word_idx += 1
            self.typing_entry.delete(0, tk.END)
            self.has_made_mistake = False
            self.render_article()
            self.auto_scroll_after_first_line_done()
            self.update_stats()
            self.update_progress_bar()

            # 當全部文章都打完時
            if self.current_word_idx >= len(self.target_words):
                self.typing_entry.config(state="disabled")
                self.update_stats(final=True)
                self.car_blink_at_finish(self.car)
                self.create_menu_bar()

    return "break"

```

賽車進度條更新機制

```

def update_progress_bar(self):
    if not hasattr(self, 'car'):
        return

    total_words = len(self.target_words)
    completed_words = len(self.typed_words)
    progress = completed_words / total_words if total_words > 0 else 0

    # 根據進度移動車子 (總長度為 finish_line_x)
    target_x = int(progress * self.finish_line_x)
    progress_text = f"{int(progress * 100)}%"

    # 取得目前car位置並設定小車動畫參數
    car_current_coords = self.track_canvas.coords(self.car)
    car_current_x = car_current_coords[0] if car_current_coords else 0
    steps = 10
    dx = (target_x - car_current_x) / steps

    def animate(step):
        if step >= steps:
            self.track_canvas.coords(self.car, target_x, 30)
            self.track_canvas.itemconfig(self.progress_text, text=progress_text)
            return
        new_x = car_current_x + dx*step
        self.track_canvas.coords(self.car, new_x, 30)
        self.track_canvas.itemconfig(self.progress_text, text=progress_text)
        self.track_canvas.after(15, lambda: animate(step + 1))
    animate(0)

```

雙人聯機 Socket 通訊溝通機制(自定義封包類型)

```

def handle_message(self, msg):
    with self.lock:
        if self.is_destroyed:
            return
        if msg["type"] == "progress":
            print("progress")
            player_id = msg["player_id"]
            name = msg["name"]
            self.players_progress[player_id] = msg["words"]
            self.add_or_update_car(player_id, name)
            # self.update_remote_car(player_id)

            if msg["finished"] and player_id not in self.finish_order:
                self.finish_order.append(player_id)
                if not self.game_over and len(self.finish_order) == len(self.players_progress):
                    self.game_over = True
                    self.show_results()

        elif msg["type"] == "introduce":
            print("introduce")
            name = msg["name"]
            player_id = msg["id"]
            self.add_or_update_car(player_id, name)
            if not self.ready_to_start:
                self.ready_to_start = True
                self.start_countdown()

        elif msg["type"] == "start":
            print("start")
            name = msg["host_player_name"]
            self.target_article: str = msg["text"]
            self.target_words = self.target_article.split()
            self.render_article()
            self.update_stats()
            self.update_all_tracks()
            self.update_all_cars()
            if not self.ready_to_start:
                self.send_json({
                    "type": "introduce",
                    "name": self.player_name,
                    "id": self.player_id,
                })
                self.ready_to_start = True
                self.start_countdown()

        elif msg["type"] == "leave":
            print("leave")
            player_id = msg["id"]
            if player_id not in self.finish_order:
                self.finish_order.append(player_id)
                self.left_players.add(player_id)
                self.mark_player_as_left(player_id)
                self.check_game_over()

```

太鼓模式 HIT 機制

```

def keypress_input(self, event):
    user_input = event.char
    if not user_input or not self.running:
        return

    # 判斷是否有打中
    for (box, text), symbol in self.beats:
        x, _ = self.canvas.coords(text)
        x_judge_line_dis = abs(x - self.judge_line)
        if x_judge_line_dis <= 45 and user_input == symbol:
            self.show_feedback("Perfect!", "#00ff88")
            self.flash_hitbox(color="#00ff88")
            self.score += 1
            self.score_label.config(text=f"Score: {self.score}")
            self.miss_label.config(text=f"Miss: {self.miss_count}")
            self.canvas.delete(text)
            self.canvas.delete(box)
            self.beats = [b for b in self.beats if b[0][1] != text]
            return
        elif x_judge_line_dis <= 100: # 在45~100範圍按下會是miss
            # 沒打中就計算miss+1
            self.show_feedback("Miss!", "#ff5555")
            self.flash_hitbox(color="#ff5555")
            self.miss_count += 1
            self.score_label.config(text=f"Score: {self.score}")
            self.miss_label.config(text=f"Miss: {self.miss_count}")
            self.beats = [b for b in self.beats if b[0][1] != text]
            self.canvas.delete(text)
            self.canvas.delete(box)
            if self.miss_count >= self.max_miss:
                self.end_game()
    return

```